Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your highest score.

Next item →

AI text generator

Overview

Previously, you built a basic AI Text generator. Now, you'll upgrade it to be more powerful and efficient. In the real world, developers choose the right tool for the job to balance speed, cost, and quality.

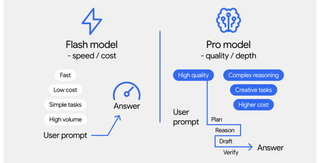
In this activity, you'll modify your text generator to act as an "AI Coach." Your coach will automatically select the best model for a user's request: the fast and efficient Flash model for quick advice, or the powerful Pro model for deep, creative analysis. You'll also enable Gemini's "thinking" feature to see the Pro model's reasoning process.

Prerequisites

- You must have a working text generator from the previous project, including a secure way to access your API key.
- **Important security reminder:** The API key must never be written directly in your code. The starting code below uses the correct, secure method to read the **GEMINI_API_KEY** from your computer's environment.

When developing, you'll need to decide the model that works best for your task. The graphic below illustrates some model trade-offs. Note that the Flash version can be used for simple tasks, but it may also be the right option for complex tasks when you need lower latency or a good balance of intelligence and speed.

Model selection trade-offs



Steps

Step 1: Start with the prerequisite code

Use the code below as your starting template. This includes the secure API key setup, which ensures you can start building immediately.

1. **Copy the entire code block** and save it as your Python file.
2. **Ensure your `GEMINI_API_KEY` is set** as an environment variable before running the script.
3. **Install dependencies:** `pip install google-genai python-dotenv`

```
from google import genai
from google.genai import types
from google.genai.types import GenerateContentConfig, ThinkingConfig


import os
from dotenv import load_dotenv

load_dotenv()  # reads .env in your project folder

def _get_api_key() -> str:
    """
    Return an API key from env vars (GEMINI_API_KEY).
    Raises if not set.
    """
    key = os.getenv("GEMINI_API_KEY")
    if not key:
        raise RuntimeError(
            "Missing API key. Set GEMINI_API_KEY "
            "(e.g., export GEMINI_API_KEY='YOUR_KEY')"
        )
    return key


def _make_client() -> genai.Client:
    """
    Create a Google GenAI client for the Developer API.
    """
    return genai.Client(api_key=_get_api_key())


client = _make_client()


def get_coaching(prompt: str, style: str = "quick"):
    """Generates a response from the AI coach based on requested style."""
    print(f"\n--- INITIATING REQUEST: {style.upper()} COACHING ---")
    # Task 2 logic will be added here
    print("Model selection logic will be added here.")
```

1. You are setting up a new AI project folder. Why is it necessary to install the Google SDK library using a specific command like `pip install` or `npm install`? Select the best answer. 1 / 1 point

- To create a unique API key for your project automatically
- To change the programming language of your file from Python to JavaScript
- To download the pre-built code functions that allow your computer to communicate with the Gemini API
- To speed up your computer's internet connection for the duration of the project

Nice work

Correct! The SDK contains the specific "translator" code that your programming language needs to send requests to and receive data from Google's servers.

Step 2: Choose the right model

Now, you will add code that chooses the model. You will update the placeholder code inside your existing `get_coaching` function.

1. **Locate the `get_coaching` function** in your file.
2. **Replace the placeholder logic** (the line `print("Model selection logic will be added here.")`) with the code below. This implements the **model selection** and adds the **visual confirmation**.

```
def get_coaching(prompt: str, style: str = "quick") -> None:
    """
    Generates a response from the AI coach, selecting model and (optionally)
    enabling thinking based on the requested style.

    style:
      - "quick":  gemini-3-flash-preview (fast, cost-efficient; thinking off)
      - "deep":   gemini-3-pro-preview  (deeper reasoning; thinking on)
    """
    print(f"\n--- INITIATING REQUEST: {style.upper()} COACHING ---")

    if style == "quick":
        model_name = "gemini-3-flash-preview"
        config = None  # thinking off by default
    elif style == "deep":
        model_name = "gemini-3-pro-preview"
        # Thinking controls live under thinking_config
        config = GenerateContentConfig(
            thinking_config=ThinkingConfig(
                thinking_level="high"
            )
        )
    else:
        raise ValueError("Invalid style. Use 'quick' or 'deep'.")

    print(f"MODEL SELECTED: {model_name}")


    # Task 3 will continue the logic here
    print("API call logic will be added here in Task 3.") # Placeholder update
```

Common mistake: Using **`gemini-3.0-pro`** for simple tasks (like "quick" coaching). This increases latency (slows the app down) and costs more money than using the efficient Flash model.

2. A developer is creating an application that has two features: a chatbot that quickly answers frequently asked questions (FAQs) and a tool that generates detailed, creative marketing slogans. To manage costs and performance, they decide to use different models for each feature. Which model selection strategy is the most effective? Select the best answer. 1 / 1 point

- Use the Flash model for the FAQ chatbot and the Pro model for the slogan generator.
- Use the Pro model for the FAQ chatbot and the Flash model for the slogan generator.
- Use the Flash model for both features to keep the application as fast and low-cost as possible.
- Use the Pro model for both features to ensure the highest quality for all responses.

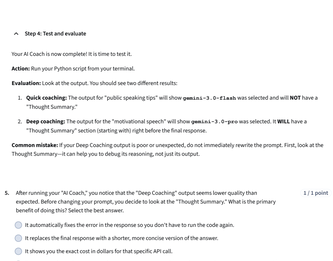
Nice work

Correct! This strategy uses the faster, lower-cost Flash model for simple, factual tasks (FAQs) and the more powerful Pro model for complex, creative tasks that require higher quality.

Step 3: Enable and analyze model "thinking" and finalize API call

Now, you will complete the function by adding the API call logic and the "thinking" feature.

API call logic + "thinking" feature



Action: Locate the placeholder logic from the previous step. Replace the placeholder logic with the complete, final API call logic below. This adds the API call, error handling, and the thinking output for the 'deep' style.

Common mistake: Enabling "thinking" for the Flash model, or for the "quick" style.

```
# The updated code for the END of the get_coaching function:

    # ... model setup from Step 2 is above this line ...

try:
    resp = client.models.generate_content(
        model=model_name,
        contents=prompt,
        config=config,
    )

    # Gemini 3 / google-genai: don't rely on "thought" parts being present.
    # If you requested thinking, you may still just get final text in resp.text.
    try:
        candidates = getattr(resp, "candidates", []) or []
        content = getattr(candidates[0], "content", None) if candidates else None
        parts = getattr(content, "parts", []) if content else []

        printed_header = False
        for part in parts:
            # Some SDK versions may expose "thought" parts; others won't.
            if getattr(part, "thought", False) and getattr(part, "text", ""):
                if not printed_header:
                    print("🧠 MODEL'S THOUGHT SUMMARY:")
                    printed_header = True
                print(part.text)

        if printed_header:
            print("-" * 20)
    except Exception:
        # Silently continue if structure differs or thoughts absent
        pass

    print("\n✅ FINAL RESPONSE:")
    print(getattr(resp, "text", ""))

except Exception as e:
    print(f"\nAn error occurred: {e}. Check your API key, network, or SDK version.")

print("-" * 30)
```

3. You are using Gemini 3 Pro (preview) to generate a complex project plan. Based on the current google-genai SDK patterns, how should you configure the API call to request enhanced reasoning ("thinking")? Select the best answer. 1 / 1 point

- Create a GenerateContentConfig with a ThinkingConfig inside it (e.g., thinking_level="high") and pass it as config=.
- Add a new parameter enable_thinking=True to the generate_content(...) call.
- Write a prompt that says "Show me your chain-of-thought before you answer."
- Set model_name to "gemini-3-pro-thinking-preview".

Nice work

Correct! With Gemini 3 in google-genai, you enable thinking by supplying a GenerateContentConfig(thinking_config=ThinkingConfig(...)) in the config parameter.

4. You have enabled Gemini 3 "thinking" for all of your API calls. You notice higher latency and higher cost. You still want useful reasoning, but need to manage performance and spend. Which parameter in ThinkingConfig allows you to control this balance? Select the best answer. 1 / 1 point

- response_mime_type
- thinking_level
- include_thoughts
- max_output_tokens

Nice work

Correct! thinking_level is used to adjust how much thinking the model performs—lower levels tend to reduce latency and cost, while higher levels increase reasoning depth.

Step 4: Test and evaluate

Your AI Coach is now complete! It is time to test it.

Action: Run your Python script from your terminal.

Evaluation: Look at the output. You should see two different results:

1. **Quick coaching:** The output for "public speaking tips" will show **`gemini-3.0-flash`** was selected and will **NOT** have a "Thought Summary."
2. **Deep coaching:** The output for the "motivational speech" will show **`gemini-3.0-pro`** was selected. It **WILL** have a "Thought Summary" section (starting with) right before the final response.

Common mistake: If your Deep Coaching output is poor or unexpected, do not immediately rewrite the prompt. First, look at the Thought Summary—it can help you to debug its reasoning, not just its output.

5. After running your "AI Coach," you notice that the "Deep Coaching" output seems lower quality than expected. Before changing your prompt, you decide to look at the "Thought Summary." What is the primary benefit of doing this? Select the best answer. 1 / 1 point

- It automatically fixes the error in the response so you don't have to run the code again.
- It replaces the final response with a shorter, more concise version of the answer.
- It shows you the exact cost in dollars for that specific API call.
- It allows you to see the model's internal reasoning and identify exactly where its logic may have gone off track.

Nice work

Correct! The Thought Summary provides transparency into the "how" and "why" of the AI's response, making it much easier to debug the reasoning process.

Success checklist

- The application successfully switches between the Pro and Flash models based on the **`style`** parameter.
- Model thinking is enabled for the Pro model requests.
- The application correctly interprets and displays the thought summary for the high-quality requests.

Key points

- **Making trade-offs:** Choosing the right model is a key skill. Use Gemini Flash for simple, high-volume tasks. Use Gemini Pro for tasks that need complex reasoning and high quality.
- **Transparency:** Model thinking gives you a window into the model's reasoning process. This is very useful for improving your prompts and understanding the AI's output for complex tasks.

Exemplar solution

(This solution shows the complete, integrated logic the learner is expected to achieve.)

```
from google import genai
from google.genai import types
from google.genai.types import GenerateContentConfig, ThinkingConfig

import os
from dotenv import load_dotenv

load_dotenv()  # reads .env in your project folder

def _get_api_key() -> str:
    """
    Return an API key from env vars (GEMINI_API_KEY fallback).
    Raises if neither is set.
    """
    key = os.getenv("GEMINI_API_KEY")
    if not key:
        raise RuntimeError(
            "Missing API key. Set GEMINI_API_KEY "
            "(e.g., export GEMINI_API_KEY='YOUR_KEY')"
        )
    return key


def _make_client() -> genai.Client:
    """
    Create a Google GenAI client for the Developer API.
    """
    return genai.Client(api_key=_get_api_key())


client = _make_client()


def get_coaching(prompt: str, style: str = "quick") -> None:
    """
    Generates a response from the AI coach, selecting model and (optionally)
    enabling thinking based on the requested style.

    style:
      - "quick":  gemini-3-flash-preview (fast, cost-efficient; thinking off)
```

Next steps

Explore how to use the thought summary internally (not just display it) to automatically generate a follow-up prompt to the model if the initial thinking indicates a possible error.